

Guide Complet Spring Boot 3.4

Starter-Kit pour Développeurs Débutants

Bill TANEKEU

Expertise technique et pédagogique pas-à-pas

Note Conceptuelle

Ce tutoriel est conçu pour un **débutant strict**. Chaque concept abstrait y est introduit par une analogie du monde réel, suivie d'une explication conceptuelle puis de son implémentation Java/Maven avec PostgreSQL.

Juin 2026

Table des matières

1	Création du Projet et Structure	3
1.1	Pourquoi un Starter-Kit ?	3
1.2	Étape 1 — L’Initialisation avec Spring Initializr	3
1.3	Étape 2 — L’Arborescence Structurée du Projet	4
2	Configuration Globale et Base de Données	6
2.1	Configuration Applicative : application.properties	6
2.1.1	La Logique Métier (Avant le code)	6
2.1.2	L’Implémentation	6
2.2	Modèles de Données et JPA (Entités)	7
2.2.1	La Logique Métier (Avant le code)	7
2.2.2	L’Implémentation de BaseEntity.java	7
2.2.3	L’Implémentation de AppUser.java	8
2.2.4	Enums de base : Role.java et AuthProvider.java	9
3	Repositories et Accès aux Données	11
3.1	Le Pattern Repository	11
3.1.1	La Logique Métier (Avant le code)	11
3.2	L’Implémentation de UserRepository.java	11
3.3	L’Implémentation de SessionRepository.java	11
4	Les DTO et la Logique Métier (Services)	13
4.1	Pourquoi des DTO (Data Transfer Objects) ?	13
4.1.1	La Logique Métier (Avant le code)	13
4.1.2	L’Implémentation des DTOs d’Authentification	13
4.2	La Couche Service et Transactions	14
4.2.1	La Logique Métier (Avant le code)	14
4.2.2	L’Implémentation de AuthService.java	14
4.2.3	L’Implémentation de SessionService.java	17
4.2.4	Entité Session : AppSession.java	18
5	Les Contrôleurs REST	20
5.1	Le Rôle du Contrôleur	20
5.1.1	La Logique Métier (Avant le code)	20
5.2	L’Implémentation de AuthController.java	20
6	Sécurité : Spring Security et Filtre JWT	22
6.1	JWT : Le Badge d’Accès d’une API Stateless	22
6.1.1	La Logique Métier (Avant le code)	22
6.1.2	La Paire : Access Token vs Refresh Token	22
6.2	L’Implémentation de JwtProperties.java	22
6.3	L’Implémentation de JwtService.java	23
6.4	L’Implémentation du Filtre de Sécurité : JwtAuthenticationFilter.java	24

6.5	L'Implémentation de UserPrincipal.java	27
6.6	La Configuration Globale : SecurityConfig.java	28
6.6.1	Points d'attention et CookieUtils	31
7	OAuth 2.0 avec Google	34
7.1	Le Fonctionnement de la Connexion Sociale	34
7.1.1	La Logique Métier (Avant le code)	34
7.2	L'Implémentation de CustomOAuth2UserService.java	34
7.3	Les utilitaires OAuth2 indispensables	36
8	Gestion du Cache Haute Performance avec Caffeine	38
8.1	Pourquoi utiliser un cache en mémoire ?	38
8.1.1	La Logique Métier (Avant le code)	38
8.2	Le ThreadLocal SessionContext	38
8.3	L'Implémentation de SessionContext.java	38
8.4	L'Implémentation de SessionInterceptor.java	39
8.5	L'Implémentation de CacheService.java	39
8.6	Configuration MVC : CacheWebConfig.java	40
9	Gestionnaire Global d'Exceptions	42
9.1	Pourquoi centraliser la gestion des erreurs ?	42
9.1.1	La Logique Métier (Avant le code)	42
9.2	L'Implémentation de AppException.java	42
9.3	L'Implémentation de GlobalExceptionHandler.java	43
10	Système de Logging (Logs)	46
10.1	Pourquoi utiliser des Logs au lieu de System.out.println ?	46
10.1.1	La Logique Métier (Avant le code)	46
10.1.2	Les Niveaux de Log (Ordre croissant de gravité)	46
10.2	Configuration dans application.properties	46
11	Test et Validation	48
11.1	Déroulement pas-à-pas du test de l'API	48
11.1.1	1. Test de l'Inscription (Register)	48
11.1.2	2. Test de l'Accès Sécurisé	48
11.1.3	3. Ajout de l'Authorization Bearer	48
11.1.4	4. Renouvellement du Token (Refresh)	48
11.1.5	5. Déconnexion (Logout)	49

Chapitre 1

Création du Projet et Structure

1.1 Pourquoi un Starter-Kit ?

Imaginez que vous construisez une maison. Au lieu d'apprendre à fabriquer vos propres briques, votre ciment et vos outils de mesure à chaque nouveau chantier, vous utilisez une **boîte à outils de chantier pré-assemblée** contenant tout le nécessaire pour démarrer immédiatement : fondations solides, arrivées d'eau, et électricité standardisée.

En développement d'applications web, un **Starter-Kit** est cette boîte à outils. Il regroupe les fonctionnalités transversales récurrentes indispensables (sécurité, authentification, cache, journalisation, gestion globale des erreurs) pour vous éviter de réinventer la roue et vous concentrer directement sur la **logique métier** (ce qui rend votre application unique).

1.2 Étape 1 — L'Initialisation avec Spring Initializr

Spring Initializr (disponible sur <https://start.spring.io>) est le configurateur officiel de projets Spring Boot. C'est l'équivalent du guichet où vous passez commande pour les composants de votre future application.

Pour notre projet `springboot-starter-kit`, configurez les options suivantes :

- **Project** : Maven Project (pour gérer nos dépendances via un fichier `pom.xml`)
- **Language** : Java
- **Spring Boot** : 3.4.5 (Version stable courante)
- **Project Metadata** :
 - Group : `com.starterkit`
 - Artifact : `springboot-starter-kit`
 - Name : `springboot-starter-kit`
 - Package name : `com.starterkit`
 - Packaging : Jar
 - Java : 21 (Version LTS récente et performante)
- **Dependencies** :
 - **Spring Web** : Pour créer des APIs REST HTTP.
 - **Spring Security** : Pour sécuriser nos routes.
 - **OAuth2 Client** : Pour l'authentification tierce (ex : Google).
 - **Spring Data JPA** : Pour l'accès et le stockage des données.
 - **PostgreSQL Driver** : Pilote de connexion à la base de données PostgreSQL.
 - **Spring Cache** : Abstraction du cache applicatif.
 - **Validation** : Validation automatique des formats de données avec des annotations (ex : `@Email`, `@NotBlank`).
 - **Lombok** : Générateur de code répétitif (getters, setters, constructeurs) au moment de la compilation.

— **Spring Boot DevTools** : Recharge automatiquement le code après modification.

Attention — Erreur fréquente

Lombok nécessite d'installer le plugin correspondant dans votre IDE (IntelliJ IDEA ou VS Code) et d'activer l'option "**Enable annotation processing**" dans les paramètres de votre éditeur de code, sinon votre IDE ne reconnaîtra pas les getters et setters générés à la volée.

1.3 Étape 2 — L'Arborescence Structurée du Projet

Une fois le projet généré et extrait, voici l'arborescence complète que nous allons mettre en place. Les fichiers indiqués avec **[Initializr]** sont créés automatiquement. Les autres sont créés par nos soins.

Listing 1.1 – Arborescence du Starter-Kit

```

1  springboot-starter-kit/
2  |-- pom.xml                                # [Initializr] Dependances et
   |   plugins Maven
3  |-- src/
4  |   |-- main/
5  |   |   |-- java/
6  |   |   |   |-- com/starterkit/
7  |   |   |   |   |-- StarterKitApplication.java # [Initializr] Point
   |   |   |   |   |   d'entree de l'application
8  |   |   |   |   |-- core/                        # Logique metier et base
9  |   |   |   |   |   |-- controller/              # Controleurs REST HTTP
10 |   |   |   |   |   |-- dto/                      # Objets de transfert de
   |   |   |   |   |   |   donnees (DTOs)
11 |   |   |   |   |   |-- entities/                 # Entites JPA de base
   |   |   |   |   |   |   (AppUser, BaseEntity)
12 |   |   |   |   |   |-- enums/                   # Enumerations (Role,
   |   |   |   |   |   |   AuthProvider)
13 |   |   |   |   |   |-- exceptions/              # Gestionnaires d'exceptions
14 |   |   |   |   |   |-- repositories/            # Interfaces d'accès aux
   |   |   |   |   |   |   donnees JPA
15 |   |   |   |   |   |-- services/                # Services metiers
16 |   |   |   |   |-- security/                    # Securite, JWT et OAuth2
17 |   |   |   |   |   |-- configs/                 # Configurations Spring
   |   |   |   |   |   |   Security
18 |   |   |   |   |   |-- entities/                # Entites specifiques a la
   |   |   |   |   |   |   securite (AppSession)
19 |   |   |   |   |   |-- models/                  # UserPrincipal (adaptateur
   |   |   |   |   |   |   UserDetails)
20 |   |   |   |   |   |-- repositories/            # Repositories dedies aux
   |   |   |   |   |   |   sessions
21 |   |   |   |   |   |-- services/                # Services de token et de
   |   |   |   |   |   |   session
22 |   |   |   |   |   |-- utils/                   # Filtres, helpers de cookies
   |   |   |   |   |   |   et oauth2
23 |   |   |   |   |-- cache/                       # Gestion du cache Caffeine
24 |   |   |   |   |   |-- configs/                 # Intercepteur MVC et
   |   |   |   |   |   |   enregistrement
25 |   |   |   |   |   |-- context/                 # ThreadLocal SessionContext
26 |   |   |   |   |   |-- interceptor/            # Intercepteur HTTP
27 |   |   |   |   |   |-- services/                # Service Caffeine Cache

```

```
28 |-- resources/  
29 |-- application.properties      # [Initializr] Configuration  
    globale
```

À retenir en 3 points

1. Un Starter-kit pose des bases réutilisables pour ne pas réinventer le socle technique à chaque projet.
2. Spring Initializr pré-configuré le projet Maven, le JDK 21 et télécharge les dépendances requises.
3. La structure en packages (core, security, cache) permet une séparation stricte des responsabilités.

Chapitre 2

Configuration Globale et Base de Données

2.1 Configuration Applicative : application.properties

2.1.1 La Logique Métier (Avant le code)

Toute application a besoin d'être paramétrée (adresse de la base de données, clés secrètes, ports d'écoute). Imaginez une boîte de dérivation électrique : c'est ici qu'on branche et relie les différents modules.

Nous utilisons le fichier `application.properties` généré par Spring Initializr. Pour la production, nous utilisons des **variables d'environnement** (ex : `${GOOGLE_CLIENT_ID}`), qui permettent de configurer l'application sans écrire les secrets en dur dans le code source (évitant ainsi de divulguer vos clés secrètes sur GitHub).

2.1.2 L'Implémentation

Ouvrez et modifiez le fichier `src/main/resources/application.properties` :

Listing 2.1 – Fichier `application.properties` complet

```
1  # application.properties
2  spring.application.name=springboot-starter-kit
3  server.port=8080
4
5  # ===== Base de donnees (PostgreSQL) =====
6  spring.datasource.driver-class-name=org.postgresql.Driver
7  # Utilise localhost en local. Le script va creer automatiquement les
   tables
8  spring.datasource.url=jdbc:postgresql://localhost:5432/starterkit_db?currentSchema=p
9  spring.datasource.username=dev_user
10 spring.datasource.password=dev_user
11 # Demande a Hibernate de mettre a jour le schema SQL de la base a
   chaque d marriage
12 spring.jpa.hibernate.ddl-auto=update
13 spring.jpa.show-sql=true
14 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
15
16 # ===== OAuth2 Google Configuration =====
17 spring.security.oauth2.client.registration.google.client-id=${GOOGLE_CLIENT_ID:your-
18 spring.security.oauth2.client.registration.google.client-secret=${GOOGLE_CLIENT_SECR
19 spring.security.oauth2.client.registration.google.redirect-uri=http://localhost:8080
20 spring.security.oauth2.client.registration.google.scope=email,profile
21
```

```

22 # ===== Custom App Security Configuration =====
23 app.auth.cookie-secret=change-this-cookie-secret-in-production
24 app.auth.access-token-cookie-max-age=900
25 app.auth.refresh-token-cookie-max-age=604800
26
27 # Redirections après connexion OAuth2 russe
28 app.oauth2.user-redirect-uri=http://localhost:5173/dashboard
29 app.oauth2.admin-redirect-uri=http://localhost:5173/admin/dashboard
30
31 # Routes accessibles publiquement
32 app.security.public-paths=/api/auth/**,/api/upload/**,/oauth2/**,/v3/api-docs/**,/swagger-ui/**
33 app.security.allowed-origins=http://localhost:5173,http://localhost:3000
34
35 # ===== Configuration JWT =====
36 # Cle secrète codée en Base64 (minimum 256 bits pour algorithme HS256)
37 jwt.secret=dGhpcy1pcy1hLXZlcnktc2VjdXJlLXN1Y3JldC1rZXktZm9yLWpzb24td2ViLXRva2Vucy0yMjA0
38 # Validez l'Access Token (ms) : 15 min
39 jwt.access-token-expiration=900000
40 # Validez le Refresh Token (ms) : 7 jours
41 jwt.refresh-token-expiration=604800000
42
43 # ===== Fichiers (Upload) =====
44 app.upload.dir=uploads
45 spring.servlet.multipart.max-file-size=100MB
46 spring.servlet.multipart.max-request-size=100MB
47
48 # ===== Cache Caffeine =====
49 spring.cache.type=caffeine
50 spring.cache.caffeine.spec=maximumSize=500,expireAfterWrite=24h

```

2.2 Modèles de Données et JPA (Entités)

2.2.1 La Logique Métier (Avant le code)

Une **Entité JPA** est une classe Java classique dont chaque instance (objet) va correspondre à une ligne dans une table SQL. L'**ORM (Object-Relational Mapping)** Hibernate va traduire automatiquement nos objets Java en requêtes SQL.

Pour des raisons professionnelles de traçabilité, nous ne voulons pas seulement stocker les informations de l'utilisateur, mais aussi savoir **quand** l'enregistrement a été créé, modifié, voire supprimé, et **par qui**. Au lieu de copier-coller ces colonnes d'audit sur chaque entité, nous créons une **Superclasse d'audit** nommée `BaseEntity`. Toutes nos futures tables hériteront d'elle (analogie : le tronc génétique commun des objets de notre base).

2.2.2 L'Implémentation de `BaseEntity.java`

Créez le fichier `src/main/java/com/starterkit/core/entities/BaseEntity.java` :

Listing 2.2 – Classe `BaseEntity.java`

```

1 package com.starterkit.core.entities;
2
3 import jakarta.persistence.Column;
4 import jakarta.persistence.MappedSuperclass;
5 import lombok.Getter;
6 import lombok.Setter;
7 import java.sql.Timestamp;
8 import java.time.Instant;

```



```

9
10 @Getter
11 @Setter
12 @MappedSuperclass // Indique a JPA que cette classe n'est pas une table
    physique, mais ses champs seront ajoutés aux classes filles
13 public abstract class BaseEntity {
14
15     @Column(name = "created_at", nullable = false, updatable = false)
16     private Timestamp createdAt = Timestamp.from(Instant.now());
17
18     @Column(name = "updated_at")
19     private Timestamp updatedAt;
20
21     @Column(name = "deleted_at")
22     private Timestamp deletedAt;
23
24     @Column(name = "created_by")
25     private String createdBy;
26
27     @Column(name = "updated_by")
28     private String updatedBy;
29
30     @Column(name = "deleted_by")
31     private String deletedBy;
32
33     @Column(name = "is_active")
34     private boolean isActive = true; // Permet une suppression logique
        (Soft Delete) au lieu d'effacer définitivement
35 }

```

2.2.3 L'Implémentation de AppUser.java

Créez le fichier `src/main/java/com/starterkit/core/entities/AppUser.java`. Cette classe modélise nos utilisateurs. Elle implémente `UserDetails` pour s'interfacer directement avec le moteur de sécurité natif de Spring Security.

Listing 2.3 – Classe AppUser.java

```

1 package com.starterkit.core.entities;
2
3 import com.starterkit.core.enums.AuthProvider;
4 import com.starterkit.core.enums.Role;
5 import jakarta.persistence.*;
6 import lombok.Getter;
7 import lombok.Setter;
8 import org.springframework.security.core.GrantedAuthority;
9 import
    org.springframework.security.core.authority.SimpleGrantedAuthority;
10 import org.springframework.security.core.userdetails.UserDetails;
11 import java.util.Collection;
12 import java.util.List;
13
14 @Entity
15 @Table(name = "sk_user") // Renommée sk_user pour éviter les conflits
    avec le mot-cle réservé SQL "user"
16 @Getter
17 @Setter
18 public class AppUser extends BaseEntity implements UserDetails {

```

```

19
20     @Id
21     @GeneratedValue(strategy = GenerationType.AUTO)
22     @Column(name = "user_id", nullable = false)
23     private Long userId;
24
25     @Column(name = "email", nullable = false, unique = true)
26     private String email;
27
28     @Column(name = "full_name")
29     private String fullName;
30
31     @Column(name = "password") // Peut etre vide si inscription 100%
32     // OAuth2 Google
33     private String password;
34
35     @Enumerated(EnumType.STRING) // Enregistre le nom du role en texte
36     // clair en base (au lieu d'un index numerique instable)
37     @Column(name = "role", nullable = false)
38     private Role role;
39
40     @Enumerated(EnumType.STRING)
41     @Column(name = "provider")
42     private AuthProvider provider; // LOCAL ou GOOGLE
43
44     @Column(name = "provider_id")
45     private String providerId; // ID unique fourni par Google
46
47     @Column(name = "profile_picture")
48     private String profilePicture;
49
50     // ===== Methodes requises par UserDetails (Spring Security) =====
51
52     @Override
53     public Collection<? extends GrantedAuthority> getAuthorities() {
54         // Retourne le role de l'utilisateur sous forme d'autorite
55         // exploitable par Spring Security
56         return List.of(new SimpleGrantedAuthority(role.name()));
57     }
58
59     @Override public String getUsername() { return email; }
60     @Override public String getPassword() { return password; }
61     @Override public boolean isAccountNonExpired() { return true; }
62     @Override public boolean isAccountNonLocked() { return true; }
63     @Override public boolean isCredentialsNonExpired() { return true; }
64     @Override public boolean isEnabled() { return isActive(); } //
65     // Relie a notre suppression logique
66 }

```

2.2.4 Enums de base : Role.java et AuthProvider.java

Créez les fichiers d'énumération dans `com/starterkit/core/enums` :

Listing 2.4 – Enum Role.java

```

1 package com.starterkit.core.enums;
2
3 public enum Role {

```

```
4     ROLE_USER ,  
5     ROLE_ADMIN  
6 }
```

Listing 2.5 – Enum AuthProvider.java

```
1 package com.starterkit.core.enums;  
2  
3 public enum AuthProvider {  
4     LOCAL ,  
5     GOOGLE  
6 }
```

À retenir en 3 points

1. La configuration par variables d'environnement dans `application.properties` évite d'exposer les secrets d'API.
2. `@MappedSuperclass` permet de factoriser les colonnes de traçabilité d'audit sans alourdir chaque entité.
3. L'implémentation de `UserDetails` par l'entité `AppUser` permet d'utiliser l'entité directement dans la sécurité Spring.

Chapitre 3

Repositories et Accès aux Données

3.1 Le Pattern Repository

3.1.1 La Logique Métier (Avant le code)

Le Repository est le **majordome** de votre base de données. Au lieu d'ouvrir des connexions JDBC manuelles et d'écrire du code SQL comme `SELECT * FROM sk_user WHERE email = ?`, vous déclarez une interface Java étendant `JpaRepository`. Spring Data JPA va analyser le nom de vos méthodes (ex : `findByEmail`) et générer **automatiquement** la requête SQL correspondante en arrière-plan.

3.2 L'Implémentation de UserRepository.java

Créez le fichier `src/main/java/com/starterkit/core/repositories/UserRepository.java` :

Listing 3.1 – Interface UserRepository.java

```
1 package com.starterkit.core.repositories;
2
3 import com.starterkit.core.entities.AppUser;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6 import java.util.Optional;
7
8 @Repository
9 public interface UserRepository extends JpaRepository<AppUser, Long> {
10
11     // Spring genere automatiquement : SELECT * FROM sk_user WHERE
12     // email = ?
13     Optional<AppUser> findByEmail(String email);
14
15     // Spring genere : SELECT CASE WHEN COUNT(*)>0 THEN true ELSE false
16     // END FROM sk_user WHERE email = ?
17     boolean existsByEmail(String email);
18
19     Optional<AppUser> findById(Long userId);
20 }
```

3.3 L'Implémentation de SessionRepository.java

Créez le fichier `src/main/java/com/starterkit/security/repositories/SessionRepository.java`. Il gère l'accès aux sessions persistantes (Refresh Tokens).

Listing 3.2 – Interface SessionRepository.java

```
1 package com.starterkit.security.repositories;
2
3 import com.starterkit.security.entities.AppSession;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6 import java.util.Optional;
7
8 @Repository
9 public interface SessionRepository extends JpaRepository<AppSession,
10     Long> {
11     Optional<AppSession> findByRefreshToken(String refreshToken);
12     Optional<AppSession> findByToken(String token);
13     void deleteAllByUserId(Long userId);
14 }
```

À retenir en 3 points

1. Les Repositories sont des interfaces qui évitent d'écrire des requêtes SQL manuelles.
2. Spring Data JPA traduit dynamiquement les noms des méthodes d'interface en requêtes SQL physiques.
3. L'utilisation d'`Optional` prévient les erreurs redoutées de type `NullPointerException`.

Chapitre 4

Les DTO et la Logique Métier (Services)

4.1 Pourquoi des DTO (Data Transfer Objects) ?

4.1.1 La Logique Métier (Avant le code)

Imaginez que vous achetez un produit en ligne. Le livreur n'a pas besoin de connaître vos données de santé, votre situation de famille ou le solde de votre compte bancaire. Il a juste besoin d'une étiquette avec votre nom et votre adresse de livraison.

Le **DTO** est cette étiquette. C'est un conteneur de données simplifié, taillé sur mesure pour la requête ou la réponse réseau. Il évite deux problèmes majeurs :

- **L'exposition de données sensibles** (ex : renvoyer le mot de passe hashé de l'entité `AppUser` dans le JSON de réponse).
- **Les attaques par injection de champs** (quand un pirate tente d'envoyer un champ `role = "ROLE_ADMIN"` dans un JSON d'inscription pour pirater le système).

En Java moderne (JDK 16+), on utilise des **records**, qui sont des classes immuables, ultra-concises et automatiques.

4.1.2 L'Implémentation des DTOs d'Authentification

Créez les fichiers de records Java suivants dans `com/starterkit/core/dto` :

Listing 4.1 – DTO d'inscription (`RegisterRequestDTO.java`)

```
1 package com.starterkit.core.dto;
2
3 import jakarta.validation.constraints.Email;
4 import jakarta.validation.constraints.NotBlank;
5 import jakarta.validation.constraints.Size;
6
7 public record RegisterRequestDTO(
8     @NotBlank(message = "L'email est obligatoire")
9     @Email(message = "Format de l'email invalide")
10    String email,
11
12    @NotBlank(message = "Le mot de passe est obligatoire")
13    @Size(min = 6, message = "Le mot de passe doit faire au moins 6
14        caracteres")
15    String password,
16
17    @NotBlank(message = "Le nom complet est obligatoire")
18    String fullName
```

```
18 ) {}
```

Listing 4.2 – DTO de connexion (LoginRequestDTO.java)

```
1 package com.starterkit.core.dto;
2
3 import jakarta.validation.constraints.Email;
4 import jakarta.validation.constraints.NotBlank;
5
6 public record LoginRequestDTO(
7     @NotBlank @Email String email,
8     @NotBlank String password
9 ) {}
```

Listing 4.3 – DTO de reponse (AuthResponseDTO.java)

```
1 package com.starterkit.core.dto;
2
3 public record AuthResponseDTO(
4     String accessToken,
5     String refreshToken,
6     long expiresIn // Exprimee en secondes pour le client frontend
7 ) {}
```

Listing 4.4 – DTO de rafraichissement (RefreshTokenRequestDTO.java)

```
1 package com.starterkit.core.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4
5 public record RefreshTokenRequestDTO(
6     @NotBlank String refreshToken
7 ) {}
```

4.2 La Couche Service et Transactions

4.2.1 La Logique Métier (Avant le code)

Le Service est le **chef de projet** de votre application. Il ne communique pas directement avec le monde extérieur (le HTTP) et ne stocke rien en direct (la base). Il orchestre les opérations : il reçoit l'ordre, appelle les repositories pour lire/écrire, crypte les mots de passe, et gère les calculs.

La transaction (@Transactional) : Imaginez un virement bancaire. Si l'argent est débité du compte A, mais qu'une coupure électrique empêche de créditer le compte B, vous perdez votre argent. Une transaction garantit que tout le bloc s'exécute avec succès ou, en cas d'erreur intermédiaire, que **toutes** les opérations SQL précédentes sont annulées (Rollback) comme si rien ne s'était produit.

4.2.2 L'Implémentation de AuthService.java

Créez le fichier `src/main/java/com/starterkit/core/services/AuthService.java` :

Listing 4.5 – Service AuthService.java

```
1 package com.starterkit.core.services;
2
3 import com.starterkit.core.dto.*;
4 import com.starterkit.core.entities.AppUser;
```

```

5  import com.starterkit.core.enums.AuthProvider;
6  import com.starterkit.core.enums.Role;
7  import com.starterkit.core.exceptions.AppException;
8  import com.starterkit.core.repositories.UserRepository;
9  import com.starterkit.security.entities.AppSession;
10 import com.starterkit.security.repositories.SessionRepository;
11 import com.starterkit.security.services.JwtService;
12 import com.starterkit.security.services.SessionService;
13 import com.starterkit.security.utils.JwtProperties;
14 import lombok.RequiredArgsConstructor;
15 import org.springframework.http.HttpStatus;
16 import
    org.springframework.security.authentication.AuthenticationManager;
17 import
    org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
18 import org.springframework.security.crypto.password.PasswordEncoder;
19 import org.springframework.stereotype.Service;
20 import org.springframework.transaction.annotation.Transactional;
21
22 @Service
23 @RequiredArgsConstructor
24 public class AuthService {
25
26     private final UserRepository userRepository;
27     private final SessionRepository sessionRepository;
28     private final SessionService sessionService;
29     private final JwtService jwtService;
30     private final PasswordEncoder passwordEncoder;
31     private final AuthenticationManager authenticationManager;
32     private final JwtProperties jwtProperties;
33
34     @Transactional // Si la creation de session echoue, le User SQL
                     // n'est pas sauvegarde
35     public AuthResponseDTO register(RegisterRequestDTO request) {
36         if (userRepository.existsByEmail(request.email())) {
37             throw new AppException("Un compte avec cet email existe
                                     deja", HttpStatus.BAD_REQUEST);
38         }
39
40         AppUser user = new AppUser();
41         user.setEmail(request.email());
42         user.setPassword(passwordEncoder.encode(request.password()));
43         // Crypte le mot de passe avant persistance
44         user.setFullName(request.fullName());
45         user.setRole(Role.ROLE_USER);
46         user.setProvider(AuthProvider.LOCAL);
47         user.setActive(true);
48         user = userRepository.save(user);
49
50         return generateTokensAndSession(user);
51     }
52
53     @Transactional
54     public AuthResponseDTO login(LoginRequestDTO request) {
55         // Authentification via Spring Security (leve
56         // BadCredentialsException si erreur)
57         authenticationManager.authenticate(
58             new

```



```

        UsernamePasswordAuthenticationToken(request.email(),
        request.password()));
57
58    AppUser user = userRepository.findByEmail(request.email())
59        .orElseThrow(() -> new AppException("Utilisateur
        introuvable", HttpStatus.NOT_FOUND));
60
61    return generateTokensAndSession(user);
62 }
63
64 @Transactional
65 public AuthResponseDTO refresh(RefreshTokenRequestDTO request) {
66     AppSession session =
        sessionRepository.findByRefreshToken(request.refreshToken())
67         .orElseThrow(() -> new AppException("Refresh token
        introuvable", HttpStatus.NOT_FOUND));
68
69     sessionService.verifyExpiration(session);
70
71     AppUser user = userRepository.findById(session.getUserId())
72         .orElseThrow(() -> new AppException("Utilisateur
        introuvable", HttpStatus.NOT_FOUND));
73
74     // Regonfle uniquement un Access Token tout neuf sans changer
        le Refresh Token
75     String newAccessToken = jwtService.generateAccessToken(user);
76     session.setToken(newAccessToken);
77     sessionRepository.save(session);
78
79     return new AuthResponseDTO(
80         newAccessToken,
81         request.refreshToken(),
82         jwtProperties.accessTokenExpiration() / 1000
83     );
84 }
85
86 @Transactional
87 public void logout(String refreshToken) {
88     AppSession session =
        sessionRepository.findByRefreshToken(refreshToken)
89         .orElseThrow(() -> new AppException("Refresh token
        introuvable", HttpStatus.NOT_FOUND));
90     session.setRevoked(true); // Revocation logique du token
91     sessionRepository.save(session);
92 }
93
94 private AuthResponseDTO generateTokensAndSession(AppUser user) {
95     String accessToken = jwtService.generateAccessToken(user);
96     String refreshToken = jwtService.generateRefreshToken();
97     sessionService.createSession(user, accessToken, refreshToken);
98     return new AuthResponseDTO(
99         accessToken,
100         refreshToken,
101         jwtProperties.accessTokenExpiration() / 1000
102     );
103 }
104 }

```

4.2.3 L'Implémentation de SessionService.java

Créez le fichier `src/main/java/com/starterkit/security/services/SessionService.java` :

Listing 4.6 – Service SessionService.java

```

1 package com.starterkit.security.services;
2
3 import com.starterkit.cache.context.SessionContext;
4 import com.starterkit.cache.services.CacheService;
5 import com.starterkit.core.entities.AppUser;
6 import com.starterkit.core.exceptions.AppException;
7 import com.starterkit.security.entities.AppSession;
8 import com.starterkit.security.repositories.SessionRepository;
9 import com.starterkit.security.utils.JwtProperties;
10 import lombok.RequiredArgsConstructor;
11 import org.slf4j.Logger;
12 import org.slf4j.LoggerFactory;
13 import org.springframework.http.HttpStatus;
14 import org.springframework.stereotype.Service;
15 import org.springframework.transaction.annotation.Transactional;
16 import java.sql.Timestamp;
17 import java.time.Instant;
18
19 @Service
20 @RequiredArgsConstructor
21 public class SessionService {
22
23     private static final Logger log =
24         LoggerFactory.getLogger(SessionService.class);
25
26     private final SessionRepository sessionRepository;
27     private final JwtProperties jwtProperties;
28     private final CacheService cacheService;
29
30     public AppSession createSession(AppUser user, String accessToken,
31                                     String refreshToken) {
32         AppSession session = new AppSession();
33         Timestamp now = Timestamp.from(Instant.now());
34         session.setUserId(user.getUserId());
35         session.setToken(accessToken);
36         session.setRefreshToken(refreshToken);
37         session.setStartAt(now);
38         session.setExpiresAt(new Timestamp(now.getTime() +
39             jwtProperties.refreshTokenExpiration()));
40         session.setRevoked(false);
41         return sessionRepository.save(session);
42     }
43
44     public AppSession getCurrentSession() {
45         // Tente de charger depuis le cache memoire ultra-rapide
46         AppSession session = cacheService.getCurrentSession();
47         if (session == null) {
48             String token = SessionContext.getToken();
49             if (token != null && !token.isBlank()) {
50                 // Sinon, va lire la base SQL lente et met a jour le
51                 // cache
52                 session = sessionRepository.findByToken(token)
53                     .orElseThrow(() -> new AppException("Session
54                         introuvable", HttpStatus.UNAUTHORIZED));
55             }
56         }
57     }
58 }

```

```

50         cacheService.putSession(token, session);
51     }
52 }
53 if (session != null && (session.isRevoked() ||
54     session.getExpiresAt().before(Timestamp.from(Instant.now()))))
55 {
56     throw new AppException("Session invalide ou expir e",
57         HttpStatus.UNAUTHORIZED);
58 }
59 return session;
60 }
61
62 @Transactional
63 public void verifyExpiration(AppSession session) {
64     if (session.isRevoked()) {
65         throw new AppException("Refresh token r voqu
66             veuillez vous reconnecter", HttpStatus.UNAUTHORIZED);
67     }
68     if
69         (session.getExpiresAt().before(Timestamp.from(Instant.now()))))
70     {
71         session.setRevoked(true);
72         sessionRepository.save(session);
73         throw new AppException("Refresh token expir      veuillez
74             vous reconnecter", HttpStatus.UNAUTHORIZED);
75     }
76 }
77
78 @Transactional
79 public void revokeAllUserSessions(AppUser user) {
80     sessionRepository.deleteAllByUserId(user.getUserId());
81     log.info("Toutes les sessions r voqu es pour userId={}",
82         user.getUserId());
83 }
84 }
85 }

```

4.2.4 Entité Session : AppSession.java

Créez le fichier d'entité de session dans `src/main/java/com/starterkit/security/entities/AppSession.java`.

Listing 4.7 – Classe AppSession.java

```

1 package com.starterkit.security.entities;
2
3 import jakarta.persistence.*;
4 import lombok.Getter;
5 import lombok.Setter;
6 import java.sql.Timestamp;
7
8 @Entity
9 @Table(name = "sk_session")
10 @Getter
11 @Setter
12 public class AppSession {
13
14     @Id
15     @GeneratedValue(strategy = GenerationType.AUTO)
16     @Column(name = "session_id")

```

```
17     private Long sessionId;
18
19     @Column(name = "user_id", nullable = false)
20     private Long userId;
21
22     @Column(name = "access_token", nullable = false, length = 2048)
23     private String token;
24
25     @Column(name = "refresh_token", nullable = false, unique = true)
26     private String refreshToken;
27
28     @Column(name = "start_at", nullable = false)
29     private Timestamp startAt;
30
31     @Column(name = "expires_at", nullable = false)
32     private Timestamp expiresAt;
33
34     @Column(name = "revoked", nullable = false)
35     private boolean revoked = false;
36 }
```

À retenir en 3 points

1. Les records Java s'utilisent comme des DTOs pour structurer proprement et protéger les échanges réseaux.
2. L'annotation `@Transactional` garantit que tout le bloc s'exécute ou qu'il s'annule complètement en cas de plantage.
3. Crypter le mot de passe via `PasswordEncoder` avant sa persistance est obligatoire pour la sécurité élémentaire.

Chapitre 5

Les Contrôleurs REST

5.1 Le Rôle du Contrôleur

5.1.1 La Logique Métier (Avant le code)

Le Contrôleur (Controller) est le **guichetier** d'une banque. Il fait face à l'extérieur. C'est lui qui réceptionne les requêtes HTTP (qui frappent à sa porte), vérifie les enveloppes (la validation avec `@Valid`), transmet l'ordre au service métier concerné, puis prépare et renvoie le JSON de réponse (le reçu HTTP `ResponseEntity`). Il doit rester léger et **dépourvu de logique métier complexe**.

5.2 L'Implémentation de `AuthController.java`

Créez le fichier `src/main/java/com/starterkit/core/controller/AuthController.java` :

Listing 5.1 – Contrôleur `AuthController.java`

```
1 package com.starterkit.core.controller;
2
3 import com.starterkit.core.dto.*;
4 import com.starterkit.core.services.AuthService;
5 import io.swagger.v3.oas.annotations.Operation;
6 import io.swagger.v3.oas.annotations.tags.Tag;
7 import jakarta.validation.Valid;
8 import lombok.RequiredArgsConstructor;
9 import org.springframework.http.ResponseEntity;
10 import org.springframework.web.bind.annotation.*;
11
12 @RestController // Dit à Spring que cette classe traite des requêtes
13                // web et retourne des réponses JSON
14 @RequestMapping("/api/auth") // Prefix de toutes les routes de cette
15                // classe
16 @RequiredArgsConstructor
17 @Tag(name = "Auth", description = "Endpoints d'authentification") //
18     // Annotation Swagger UI
19 public class AuthController {
20
21     private final AuthService authService;
22
23     @PostMapping("/register")
24     @Operation(summary = "Inscription d'un nouvel utilisateur")
25     public ResponseEntity<AuthResponseDTO> register(@Valid @RequestBody
26         RegisterRequestDTO request) {
```

```
23      // @Valid declenche la validation des regles du DTO (comme
24      // @NotBlank ou @Email)
25      return ResponseEntity.ok(authService.register(request));
26  }
27
28  @PostMapping("/login")
29  @Operation(summary = "Connexion avec email + mot de passe")
30  public ResponseEntity<AuthResponseDTO> login(@Valid @RequestBody
31      LoginRequestDTO request) {
32      return ResponseEntity.ok(authService.login(request));
33  }
34
35  @PostMapping("/refresh")
36  @Operation(summary = "Renouvellement de l'access token")
37  public ResponseEntity<AuthResponseDTO> refresh(@Valid @RequestBody
38      RefreshTokenRequestDTO request) {
39      return ResponseEntity.ok(authService.refresh(request));
40  }
41
42  @PostMapping("/logout")
43  @Operation(summary = "Déconnexion et invalide le refresh token")
44  public ResponseEntity<Void> logout(@Valid @RequestBody
45      RefreshTokenRequestDTO request) {
46      authService.logout(request.refreshToken());
47      return ResponseEntity.noContent().build(); // Retourne un
48          statut HTTP 204 No Content
49  }
50 }
```

À retenir en 3 points

1. Un contrôleur REST ne doit jamais porter la logique métier, il délègue tout aux Services.
2. L'annotation `@Valid` garantit la conformité des données clientes reçues avant leur traitement.
3. La classe `ResponseEntity` permet d'envelopper la réponse avec des codes HTTP appropriés (200 OK, 204 No Content).

Chapitre 6

Sécurité : Spring Security et Filtre JWT

6.1 JWT : Le Badge d'Accès d'une API Stateless

6.1.1 La Logique Métier (Avant le code)

Dans une application web traditionnelle, le serveur mémorise chaque client connecté dans sa propre mémoire (mécanisme de Session Stateful). Cela pose problème si vous avez des milliers d'utilisateurs ou si vous répartissez la charge sur plusieurs serveurs.

Pour résoudre cela, nous créons un système **Stateless** avec **JWT (JSON Web Token)**.

Note Conceptuelle

Analogie du Bracelet de Festival : Imaginez que vous entrez dans un festival de musique. Au guichet d'entrée, vous présentez vos papiers et payez votre billet (Phase de Login). Le guichetier vous agrafe alors un **bracelet inviolable** portant votre nom et la mention "Zone VIP" (le JWT, signé avec la clé secrète du festival).

Désormais, pour chaque stand de boisson ou de concert où vous vous présentez, le vigile n'a pas besoin de téléphoner au guichet pour demander si vous êtes bien inscrit sur la liste (Stateless). Il regarde juste votre bracelet, vérifie que la signature du festival est authentique et valide, puis vous laisse entrer.

6.1.2 La Paire : Access Token vs Refresh Token

Un token volé est un laissez-passer pour un pirate. Pour limiter les risques, nous utilisons deux tokens :

- **L'Access Token** : C'est le bracelet à votre poignet. Sa validité est très courte (ex : 15 minutes). S'il est volé, le pirate perd l'accès très rapidement.
- **Le Refresh Token** : C'est un reçu papier aléatoire stocké dans votre portefeuille (ou en base de données sécurisée). Il a une validité longue (ex : 7 jours). Quand le bracelet expire, le client présente le reçu pour en obtenir un nouveau sans repasser par le processus complet d'authentification.

6.2 L'Implémentation de JwtProperties.java

Créez le fichier `src/main/java/com/starterkit/security/utils/JwtProperties.java` :

Listing 6.1 – Propriétés JwtProperties.java

```
1 package com.starterkit.security.utils;
```

```

2
3 import
    org.springframework.boot.context.properties.ConfigurationProperties;
4
5 @ConfigurationProperties(prefix = "jwt") // Lie automatiquement les
    proprietes du fichier properties debutant par "jwt."
6 public record JwtProperties(
7     String secret,
8     long accessTokenExpiration,
9     long refreshTokenExpiration
10 ) {}

```

6.3 L'Implémentation de JwtService.java

Créez le fichier `src/main/java/com/starterkit/security/services/JwtService.java` :

Listing 6.2 – Service JwtService.java

```

1 package com.starterkit.security.services;
2
3 import com.starterkit.core.entities.AppUser;
4 import com.starterkit.security.models.UserPrincipal;
5 import com.starterkit.security.utils.JwtProperties;
6 import io.jsonwebtoken.Claims;
7 import io.jsonwebtoken.Jwts;
8 import io.jsonwebtoken.security.Keys;
9 import lombok.RequiredArgsConstructor;
10 import org.springframework.stereotype.Service;
11 import javax.crypto.SecretKey;
12 import java.util.Base64;
13 import java.util.Date;
14 import java.util.UUID;
15 import java.util.function.Function;
16
17 @Service
18 @RequiredArgsConstructor
19 public class JwtService {
20
21     private final JwtProperties jwtProperties;
22
23     public String generateAccessToken(AppUser user) {
24         return buildAccessToken(user.getEmail(), user.getRole().name());
25     }
26
27     public String generateAccessToken(UserPrincipal principal) {
28         return buildAccessToken(principal.getEmail(),
29             principal.getRole().name());
30     }
31
32     public String generateRefreshToken() {
33         return UUID.randomUUID().toString();
34     }
35
36     public String extractEmail(String token) {
37         return extractClaim(token, Claims::getSubject);
38     }
39
40     public Date extractExpiration(String token) {

```



```

40         return extractClaim(token, Claims::getExpiration);
41     }
42
43     public <T> T extractClaim(String token, Function<Claims, T>
44         resolver) {
45         return resolver.apply(extractAllClaims(token));
46     }
47
48     public boolean isTokenValid(String token, AppUser user) {
49         String email = extractEmail(token);
50         return email.equals(user.getEmail()) && !isTokenExpired(token);
51     }
52
53     private String buildAccessToken(String email, String role) {
54         long now = System.currentTimeMillis();
55         return Jwts.builder()
56             .subject(email)
57             .claim("roles", role) // Stocke le role de
58                                 // l'utilisateur dans les claims du token
59             .issuedAt(new Date(now))
60             .expiration(new Date(now +
61                 jwtProperties.accessTokenExpiration()))
62             .signWith(getSigningKey()) // Signe le token avec notre
63                                     // cl HMAC SHA256 secrete
64             .compact();
65     }
66
67     private Claims extractAllClaims(String token) {
68         return Jwts.parser()
69             .verifyWith(getSigningKey()) // Verifie l'authenticite
70                                         // de la signature
71             .build()
72             .parseSignedClaims(token)
73             .getPayload();
74     }
75
76     private boolean isTokenExpired(String token) {
77         return extractExpiration(token).before(new Date());
78     }
79
80     private SecretKey getSigningKey() {
81         byte[] keyBytes =
82             Base64.getDecoder().decode(jwtProperties.secret());
83         return Keys.hmacShaKeyFor(keyBytes);
84     }
85 }

```

6.4 L'Implémentation du Filtre de Sécurité : JwtAuthenticationFilter.java

Le filtre est la barrière de contrôle. À chaque requête arrivant sur notre serveur, ce filtre intercepte le token, l'examine, extrait l'utilisateur, et l'injecte dans le contexte de sécurité s'il est valide.

Créez le fichier `src/main/java/com/starterkit/security/utils/JwtAuthenticationFilter.java`:

Listing 6.3 – Filtre JwtAuthenticationFilter.java

```

1  package com.starterkit.security.utils;
2
3  import com.starterkit.core.entities.AppUser;
4  import com.starterkit.core.repositories.UserRepository;
5  import com.starterkit.security.services.JwtService;
6  import jakarta.servlet.FilterChain;
7  import jakarta.servlet.ServletException;
8  import jakarta.servlet.http.HttpServletRequest;
9  import jakarta.servlet.http.HttpServletResponse;
10 import lombok.RequiredArgsConstructor;
11 import org.slf4j.Logger;
12 import org.slf4j.LoggerFactory;
13 import org.springframework.lang.NonNull;
14 import
    org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
15 import org.springframework.security.core.context.SecurityContextHolder;
16 import
    org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
17 import org.springframework.stereotype.Component;
18 import org.springframework.web.filter.OncePerRequestFilter;
19 import java.io.IOException;
20 import java.util.Set;
21
22 @Component
23 @RequiredArgsConstructor
24 public class JwtAuthenticationFilter extends OncePerRequestFilter {
25
26     private static final Logger log =
27         LoggerFactory.getLogger(JwtAuthenticationFilter.class);
28
29     // Routes publiques à ignorer immédiatement pour optimiser les
30     // performances
31     private static final Set<String> BYPASS_PATHS = Set.of(
32         "/api/auth/login",
33         "/api/auth/register",
34         "/api/auth/refresh",
35         "/api/auth/logout"
36     );
37
38     private final JwtService jwtService;
39     private final UserRepository userRepository;
40
41     @Override
42     protected void doFilterInternal(
43         @NonNull HttpServletRequest request,
44         @NonNull HttpServletResponse response,
45         @NonNull FilterChain chain) throws ServletException,
46         IOException {
47
48         if (BYPASS_PATHS.contains(request.getRequestURI())) {
49             chain.doFilter(request, response);
50             return;
51         }
52
53         String jwt = extractToken(request);
54         if (jwt == null) {
55             chain.doFilter(request, response);
56             return;
57         }
58     }
59 }

```

```

54     }
55
56     try {
57         String email;
58         try {
59             email = jwtService.extractEmail(jwt);
60         } catch (Exception e) {
61             log.warn("JWT invalide ou expire : {}", e.getMessage());
62             sendUnauthorized(response, "Token invalide ou expire");
63             return;
64         }
65
66         if (email != null &&
67             SecurityContextHolder.getContext().getAuthentication()
68             == null) {
69             AppUser user =
70                 userRepository.findByEmail(email).orElse(null);
71             if (user == null) {
72                 log.warn("JWT valide mais utilisateur introuvable :
73                     {}", email);
74                 sendUnauthorized(response, "Utilisateur
75                     introuvable");
76                 return;
77             }
78             if (!jwtService.isTokenValid(jwt, user)) {
79                 log.warn("Token expire pour : {}", email);
80                 sendUnauthorized(response, "Token expire");
81                 return;
82             }
83
84             // Genere l'adaptateur principal
85             com.starterkit.security.models.UserPrincipal principal =
86                 com.starterkit.security.models.UserPrincipal.create(user);
87
88             // Emballe le principal dans un objet
89             // d'authentification reconnu par Spring
90             UsernamePasswordAuthenticationToken authToken =
91                 new
92                     UsernamePasswordAuthenticationToken(principal,
93                     null, principal.getAuthorities());
94             authToken.setDetails(new
95                 WebAuthenticationDetailsSource().buildDetails(request));
96
97             // Enregistre l'utilisateur authentifie dans le Thread
98             // local de securite courant
99             SecurityContextHolder.getContext().setAuthentication(authToken);
100         }
101
102         chain.doFilter(request, response);
103     } catch (Exception e) {
104         log.error("Erreur inattendue dans le filtre JWT : {}",
105             e.getMessage(), e);
106         sendUnauthorized(response, "Erreur lors de la validation du
107             token");
108     }
109 }

```

```

100     private String extractToken(HttpServletRequest request) {
101         String header = request.getHeader("Authorization");
102         if (header != null && header.startsWith("Bearer ")) {
103             return header.substring(7); // Coupe la chaîne pour ignorer
104                                     // "Bearer "
105         }
106         // Fallback sur le cookie (utile pour l'OAuth2 ou le SSR)
107         return CookieUtils.getCookie(request, "accessToken")
108             .map(c -> c.getValue())
109             .orElse(null);
110     }
111     private void sendUnauthorized(HttpServletRequest response, String
112         message) throws IOException {
113         response.setStatus(HttpStatus.UNAUTHORIZED);
114         response.setContentType("application/json");
115         response.setCharacterEncoding("UTF-8");
116         response.getWriter().write("{\"error\": \"" + message + "\",
117                                     \"status\": 401}");
118     }
119 }

```

6.5 L'Implémentation de UserPrincipal.java

Créez le fichier `src/main/java/com/starterkit/security/models/UserPrincipal.java` :

Listing 6.4 – Classe UserPrincipal.java

```

1  package com.starterkit.security.models;
2
3  import com.starterkit.core.entities.AppUser;
4  import com.starterkit.core.enums.Role;
5  import lombok.Getter;
6  import org.springframework.security.core.GrantedAuthority;
7  import org.springframework.security.core.authority.SimpleGrantedAuthority;
8  import org.springframework.security.core.userdetails.UserDetails;
9  import org.springframework.security.oauth2.core.user.OAuth2User;
10 import java.util.Collection;
11 import java.util.List;
12 import java.util.Map;
13
14 @Getter
15 public class UserPrincipal implements UserDetails, OAuth2User {
16
17     private final Long id;
18     private final String email;
19     private final String password;
20     private final Role role;
21     private Map<String, Object> attributes; // Pour stocker les
22                                         // données brutes renvoyées par Google
23
24     private UserPrincipal(AppUser user) {
25         this.id = user.getId();
26         this.email = user.getEmail();
27         this.password = user.getPassword();
28         this.role = user.getRole();
29     }

```

```

29
30     public static UserPrincipal create(AppUser user) {
31         return new UserPrincipal(user);
32     }
33
34     public static UserPrincipal create(AppUser user, Map<String,
35         Object> attributes) {
36         UserPrincipal principal = new UserPrincipal(user);
37         principal.attributes = attributes;
38         return principal;
39     }
40
41     @Override public String getUsername() { return email; }
42     @Override public String getPassword() { return password; }
43     @Override public boolean isAccountNonExpired() { return true; }
44     @Override public boolean isAccountNonLocked() { return true; }
45     @Override public boolean isCredentialsNonExpired() { return true; }
46     @Override public boolean isEnabled() { return true; }
47
48     @Override
49     public Collection<? extends GrantedAuthority> getAuthorities() {
50         return List.of(new SimpleGrantedAuthority(role.name()));
51     }
52
53     @Override public Map<String, Object> getAttributes() { return
54         attributes; }
55     @Override public String getName() { return email; }
56 }

```

6.6 La Configuration Globale : SecurityConfig.java

Ce fichier coordonne tout l'écosystème de sécurité. Il indique quelles routes sont publiques, quelles routes requièrent quel rôle, active le mode stateless et configure le traitement OAuth2.

Créez le fichier `src/main/java/com/starterkit/security/configs/SecurityConfig.java` :

Listing 6.5 – Configuration centrale de la securite SecurityConfig.java

```

1  package com.starterkit.security.configs;
2
3  import com.starterkit.core.repositories.UserRepository;
4  import
5      com.starterkit.security.repositories.HttpCookieOAuth2AuthorizationRequestRepository;
6  import com.starterkit.security.services.CustomOAuth2UserService;
7  import com.starterkit.security.utils.JwtAuthenticationFilter;
8  import com.starterkit.security.utils.JwtProperties;
9  import com.starterkit.security.utils.OAuth2AuthenticationFailureHandler;
10 import com.starterkit.security.utils.OAuth2AuthenticationSuccessHandler;
11 import org.springframework.beans.factory.annotation.Value;
12 import
13     org.springframework.boot.context.properties.EnableConfigurationProperties;
14 import org.springframework.context.annotation.Bean;
15 import org.springframework.context.annotation.Configuration;
16 import org.springframework.http.HttpStatus;
17 import
18     org.springframework.security.authentication.AuthenticationManager;
19 import
20     org.springframework.security.authentication.dao.DaoAuthenticationProvider;

```

```

17 import
    org.springframework.security.config.annotation.authentication.configuration.Authen
18 import
    org.springframework.security.config.annotation.method.configuration.EnableMethodS
19 import
    org.springframework.security.config.annotation.web.builders.HttpSecurity;
20 import
    org.springframework.security.config.annotation.web.configuration.EnableWebSecurit
21 import
    org.springframework.security.config.annotation.web.configurers.AbstractHttpConfig
22 import org.springframework.security.config.http.SessionCreationPolicy;
23 import org.springframework.security.core.userdetails.UserDetailsService;
24 import
    org.springframework.security.core.userdetails.UsernameNotFoundException;
25 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
26 import org.springframework.security.crypto.password.PasswordEncoder;
27 import org.springframework.security.web.SecurityFilterChain;
28 import
    org.springframework.security.web.authentication.HttpStatusEntryPoint;
29 import
    org.springframework.security.web.authentication.UsernamePasswordAuthenticationFil
30 import org.springframework.web.cors.CorsConfiguration;
31 import org.springframework.web.cors.CorsConfigurationSource;
32 import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
33 import java.util.Arrays;
34 import java.util.List;
35
36 @Configuration
37 @EnableWebSecurity
38 @EnableMethodSecurity // Permet de proteger les services par annotation
    (ex: @PreAuthorize)
39 @EnableConfigurationProperties(JwtProperties.class)
40 public class SecurityConfig {
41
42     private final JwtAuthenticationFilter jwtAuthenticationFilter;
43     private final CustomOAuth2UserService customOAuth2UserService;
44     private final OAuth2AuthenticationSuccessHandler successHandler;
45     private final OAuth2AuthenticationFailureHandler failureHandler;
46     private final HttpCookieOAuth2AuthorizationRequestRepository
        cookieRepo;
47
48     @Value("${app.security.public-paths}")
49     private String[] publicPaths;
50
51     @Value("${app.security.allowed-origins}")
52     private String allowedOrigins;
53
54     public SecurityConfig(JwtAuthenticationFilter
        jwtAuthenticationFilter,
55                          CustomOAuth2UserService
        customOAuth2UserService,
56                          OAuth2AuthenticationSuccessHandler
        successHandler,
57                          OAuth2AuthenticationFailureHandler
        failureHandler,
58                          HttpCookieOAuth2AuthorizationRequestRepository
        cookieRepo) {
59         this.jwtAuthenticationFilter = jwtAuthenticationFilter;

```

```

60         this.customOAuth2UserService = customOAuth2UserService;
61         this.successHandler           = successHandler;
62         this.failureHandler           = failureHandler;
63         this.cookieRepo               = cookieRepo;
64     }
65
66     @Bean
67     public SecurityFilterChain securityFilterChain(HttpSecurity http)
68         throws Exception {
69         http
70             .csrf(AbstractHttpConfigurer::disable) // Desactivé car
71             // l'application utilise des tokens et est stateless
72             .sessionManagement(s ->
73                 s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
74             // Pas de session HTTP classique
75             .cors(cors ->
76                 cors.configurationSource(corsConfigurationSource()))
77             .authorizeHttpRequests(auth -> auth
78                 .requestMatchers(publicPaths).permitAll() // Routes
79                 // publiques définies dans application.properties
80                 .anyRequest().authenticated()
81             )
82             .exceptionHandling(ex -> ex
83                 .defaultAuthenticationEntryPointFor(
84                     new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED),
85                     // Si accès refusé sur l'API, retourne 401 au
86                     // lieu d'une page HTML de login
87                     req -> req.getServletPath().startsWith("/api/")
88                 )
89             )
90             .oauth2Login(oauth2 -> oauth2
91                 .authorizationEndpoint(a -> a
92                     .baseUrl("/oauth2/authorize") // Route pour initier
93                     // la connexion Google
94                     .authorizationRequestRepository(cookieRepo)) //
95                     // Stocke les requêtes d'auth temporaires dans les
96                     // cookies
97                 .redirectEndpoint(r -> r
98                     .baseUrl("/oauth2/callback/*")) // Route de retour
99                     // Google
100                 .userInfoEndpoint(u -> u
101                     .userService(customOAuth2UserService)) // Service
102                     // de traitement des données reçues
103                 .successHandler(successHandler)
104                 .failureHandler(failureHandler)
105             )
106             // Injecte notre validateur de token personnalis avant le
107             // validateur par défaut de Spring
108             .addFilterBefore(jwtAuthenticationFilter,
109                 UsernamePasswordAuthenticationFilter.class);
110
111         return http.build();
112     }
113
114     @Bean
115     public AuthenticationManager
116         authenticationManager(AuthenticationConfiguration config) throws
117         Exception {

```

```

101     return config.getAuthenticationManager();
102 }
103
104 @Bean
105 public DaoAuthenticationProvider
106     authenticationProvider(UserDetailsService uds) {
107     DaoAuthenticationProvider provider = new
108         DaoAuthenticationProvider();
109     provider.setUserDetailsService(uds);
110     provider.setPasswordEncoder(passwordEncoder());
111     return provider;
112 }
113
114 @Bean
115 public UserDetailsService userDetailsService(UserRepository
116     userRepository) {
117     return username -> userRepository.findByEmail(username)
118         .orElseThrow(() -> new
119             UsernameNotFoundException("Utilisateur introuvable :
120                 " + username));
121 }
122
123 @Bean
124 public PasswordEncoder passwordEncoder() {
125     return new BCryptPasswordEncoder(10); // Algorithme BCrypt fort
126         avec un facteur de securite (Rounds) gal a 10
127 }
128
129 @Bean
130 public CorsConfigurationSource corsConfigurationSource() {
131     CorsConfiguration config = new CorsConfiguration();
132     config.setAllowedOrigins(Arrays.asList(allowedOrigins.split(",")));
133     config.setAllowedMethods(List.of("GET", "POST", "PUT",
134         "DELETE", "OPTIONS", "PATCH"));
135     config.setAllowedHeaders(List.of("*"));
136     config.setAllowCredentials(true); // Indispensable pour
137         autoriser l'echange automatique de cookies securises
138     config.setExposedHeaders(List.of("Authorization",
139         "Content-Type"));
140     config.setMaxAge(3600L);
141
142     UrlBasedCorsConfigurationSource source = new
143         UrlBasedCorsConfigurationSource();
144     source.registerCorsConfiguration("/api/**", config);
145     source.registerCorsConfiguration("/uploads/**", config);
146     return source;
147 }
148 }

```

6.6.1 Points d'attention et CookieUtils

Pour des raisons de commodité, créez la classe utilitaire `CookieUtils.java` dans `com/starterkit/security/`

Listing 6.6 – Classe `CookieUtils.java`

```

1 package com.starterkit.security.utils;
2
3 import jakarta.servlet.http.Cookie;

```



```

4 import jakarta.servlet.http.HttpServletRequest;
5 import jakarta.servlet.http.HttpServletResponse;
6 import org.springframework.util.SerializationUtils;
7 import java.util.Base64;
8 import java.util.Optional;
9
10 public class CookieUtils {
11
12     public static Optional<Cookie> getCookie(HttpServletRequest
        request, String name) {
13         Cookie[] cookies = request.getCookies();
14         if (cookies != null) {
15             for (Cookie cookie : cookies) {
16                 if (cookie.getName().equals(name)) return
                    Optional.of(cookie);
17             }
18         }
19         return Optional.empty();
20     }
21
22     public static Cookie getAccessTokenCookie(String token, long
        maxAge) {
23         Cookie cookie = new Cookie("accessToken", token);
24         cookie.setPath("/");
25         cookie.setHttpOnly(true); // Interdit l'accès en Javascript
            (bloque les failles XSS)
26         cookie.setMaxAge((int) maxAge);
27         return cookie;
28     }
29
30     public static Cookie getRefreshTokenCookie(String token, long
        maxAge) {
31         Cookie cookie = new Cookie("refreshToken", token);
32         cookie.setPath("/");
33         cookie.setHttpOnly(true);
34         cookie.setMaxAge((int) maxAge);
35         return cookie;
36     }
37
38     public static void deleteCookie(HttpServletRequest request,
        HttpServletResponse response, String name) {
39         Cookie[] cookies = request.getCookies();
40         if (cookies != null) {
41             for (Cookie cookie : cookies) {
42                 if (cookie.getName().equals(name)) {
43                     cookie.setValue("");
44                     cookie.setPath("/");
45                     cookie.setMaxAge(0);
46                     response.addCookie(cookie);
47                 }
48             }
49         }
50     }
51
52     public static String serialize(Object object) {
53         return
            Base64.getUrlEncoder().encodeToString(SerializationUtils.serialize(object));
54     }

```

```
55
56     public static <T> T deserialize(Cookie cookie, Class<T> cls) {
57         return
58             cls.cast(SerializationUtils.deserialize(Base64.getUrlDecoder().decode(cookie.getValue())));
59     }
```

À retenir en 3 points

1. Le JWT permet un fonctionnement "stateless" décorrélé des sessions mémoires lourdes du serveur.
2. L'Access Token (court) et le Refresh Token (long) gèrent la sécurité en cas d'interception réseau.
3. Configurer des cookies en mode **HttpOnly** empêche les scripts malveillants d'accéder aux jetons via Javascript.

Chapitre 7

OAuth 2.0 avec Google

7.1 Le Fonctionnement de la Connexion Sociale

7.1.1 La Logique Métier (Avant le code)

Permettre à un utilisateur de se connecter avec Google évite de lui faire saisir un énième formulaire d'inscription et de mémoriser un nouveau mot de passe.

Mécanisme OAuth2 :

1. L'utilisateur clique sur "Se connecter avec Google".
2. Le frontend le redirige vers l'API de Google (avec notre Identifiant Client).
3. L'utilisateur s'identifie chez Google et accepte de partager son Email et son Profil public.
4. Google redirige l'utilisateur vers notre application serveur avec un **Code d'autorisation**.
5. Notre serveur attrape ce code, contacte Google en coulisses pour l'échanger contre les données de l'utilisateur (Email, Nom, Photo).
6. Nous cherchons l'utilisateur en base. S'il n'existe pas, nous l'inscrivons à la volée. S'il existe, nous le mettons à jour.
7. Enfin, nous créons sa session locale et lui retournons nos propres JWTs (Access/Refresh).

7.2 L'Implémentation de CustomOAuth2UserService.java

Créez le fichier `src/main/java/com/starterkit/security/services/CustomOAuth2UserService.java` :

Listing 7.1 – Traitement utilisateur OAuth2 CustomOAuth2UserService.java

```
1 package com.starterkit.security.services;
2
3 import com.starterkit.core.entities.AppUser;
4 import com.starterkit.core.enums.AuthProvider;
5 import com.starterkit.core.enums.Role;
6 import com.starterkit.core.repositories.UserRepository;
7 import com.starterkit.security.models.UserPrincipal;
8 import com.starterkit.security.utils.OAuth2UserInfo;
9 import com.starterkit.security.utils.OAuth2UserInfoFactory;
10 import lombok.RequiredArgsConstructor;
11 import
    org.springframework.security.oauth2.client.userinfo.DefaultOAuth2UserService;
12 import
    org.springframework.security.oauth2.client.userinfo.OAuth2UserRequest;
13 import
    org.springframework.security.oauth2.core.OAuth2AuthenticationException;
14 import org.springframework.security.oauth2.core.user.OAuth2User;
```

```

15 import org.springframework.stereotype.Service;
16 import org.springframework.util.StringUtils;
17
18 @Service
19 @RequiredArgsConstructor
20 public class CustomOAuth2UserService extends DefaultOAuth2UserService {
21
22     private final UserRepository userRepository;
23
24     @Override
25     public OAuth2User loadUser(OAuth2UserRequest request) throws
        OAuth2AuthenticationException {
26         // Appelle le service par défaut de Spring pour récupérer les
           // données brutes depuis Google
27         OAuth2User oAuth2User = super.loadUser(request);
28         String registrationId =
           request.getClientRegistration().getRegistrationId();
29
30         // Convertit les attributs bruts Google selon notre modèle
           // standardise
31         OAuth2UserInfo userInfo =
           OAuth2UserInfoFactory.getOAuth2UserInfo(registrationId,
           oAuth2User.getAttributes());
32
33         if (!StringUtils.hasText(userInfo.getEmail())) {
34             throw new OAuth2AuthenticationException("Email introuvable
           chez le provider OAuth2");
35         }
36
37         // Tente de trouver l'utilisateur en base, le met à jour s'il
           // existe, sinon le crée
38         AppUser user = userRepository.findByEmail(userInfo.getEmail())
           .map(existing -> updateUser(existing, userInfo))
           .orElseGet(() -> createUser(registrationId, userInfo));
39
40         return UserPrincipal.create(user, oAuth2User.getAttributes());
41     }
42
43     private AppUser createUser(String registrationId, OAuth2UserInfo
        info) {
44         AppUser user = new AppUser();
45         user.setProvider(AuthProvider.valueOf(registrationId.toUpperCase()));
46         user.setProviderId(info.getId());
47         user.setEmail(info.getEmail());
48         user.setFullName(info.getName());
49         user.setProfilePicture(info.getPicture());
50         user.setRole(Role.ROLE_USER);
51         user.setActive(true);
52         return userRepository.save(user);
53     }
54
55     private AppUser updateUser(AppUser user, OAuth2UserInfo info) {
56         user.setFullName(info.getName());
57         user.setProfilePicture(info.getPicture());
58         return userRepository.save(user);
59     }
60 }
61
62 }

```

7.3 Les utilitaires OAuth2 indispensables

Créez la fabrique d'informations utilisateur dans `src/main/java/com/starterkit/security/Utils/OAuth2`

Listing 7.2 – Classe OAuth2UserInfoFactory.java

```

1 package com.starterkit.security.utils;
2
3 import com.starterkit.core.enums.AuthProvider;
4 import java.util.Map;
5
6 public class OAuth2UserInfoFactory {
7     public static OAuth2UserInfo getOAuth2UserInfo(String
8         registrationId, Map<String, Object> attributes) {
9         if
10             (registrationId.equalsIgnoreCase(AuthProvider.GOOGLE.toString()))
11             {
12                 return new GoogleOAuth2UserInfo(attributes);
13             } else {
14                 throw new IllegalArgumentException("Provider non supporte :
15                     " + registrationId);
16             }
17     }
18 }

```

Créez le modèle Google associé dans `src/main/java/com/starterkit/security/Utils/GoogleOAuth2User`

Listing 7.3 – Classe GoogleOAuth2UserInfo.java

```

1 package com.starterkit.security.utils;
2
3 import java.util.Map;
4
5 public class GoogleOAuth2UserInfo extends OAuth2UserInfo {
6     public GoogleOAuth2UserInfo(Map<String, Object> attributes) {
7         super(attributes); }
8     @Override public String getId() { return (String)
9         attributes.get("sub"); }
10    @Override public String getName() { return (String)
11        attributes.get("name"); }
12    @Override public String getEmail() { return (String)
13        attributes.get("email"); }
14    @Override public String getPicture() { return (String)
15        attributes.get("picture"); }
16 }

```

Et la classe abstraite OAuth2UserInfo.java :

Listing 7.4 – Classe OAuth2UserInfo.java

```

1 package com.starterkit.security.utils;
2
3 import java.util.Map;
4
5 public abstract class OAuth2UserInfo {
6     protected Map<String, Object> attributes;
7     public OAuth2UserInfo(Map<String, Object> attributes) {
8         this.attributes = attributes; }
9     public abstract String getId();
10    public abstract String getName();
11    public abstract String getEmail();

```

```
11     public abstract String getPicture();  
12 }
```

À retenir en 3 points

1. L'OAuth2 déporte le processus d'authentification vers des tiers de confiance (ex : Google).
2. La méthode `loadUser` intercepte les infos reçues pour synchroniser notre base de données locale.
3. La modélisation avec `OAuth2UserInfo` permet d'ajouter facilement d'autres providers (GitHub, Apple...).

Chapitre 8

Gestion du Cache Haute Performance avec Caffeine

8.1 Pourquoi utiliser un cache en mémoire ?

8.1.1 La Logique Métier (Avant le code)

Imaginez que vous lisez un livre. Chaque fois que vous voulez vous rappeler d'un détail de l'intrigue, au lieu de relire toutes les pages depuis le début en montant à la bibliothèque au grenier (Requête SQL lente en base de données PostgreSQL), vous écrivez cette information sur une **petite fiche cartonnée posée sur votre table de chevet** (le Cache en mémoire Caffeine). L'accès y est instantané.

Dans notre Starter-kit, pour chaque requête sécurisée d'un utilisateur, l'application doit valider la session. Faire une requête SQL de session à chaque seconde ralentirait considérablement l'API. Nous stockons donc les sessions actives dans un cache Caffeine haute performance.

8.2 Le ThreadLocal SessionContext

Dans un serveur web, chaque requête HTTP entrante est affectée à un **Thread** (un canal d'exécution dédié). Nous voulons que la session de l'utilisateur soit consultable partout (Contrôleurs, Services) sans devoir la faire transiter en argument dans toutes les signatures de nos méthodes Java. Nous utilisons un **ThreadLocal** qui sert de "casier personnel" attaché au thread le temps de la requête.

8.3 L'Implémentation de SessionContext.java

Créez le fichier `src/main/java/com/starterkit/cache/context/SessionContext.java` :

Listing 8.1 – Classe SessionContext.java

```
1 package com.starterkit.cache.context;
2
3 public class SessionContext {
4     // Le ThreadLocal garantit que chaque thread poss de sa propre
5     // variable isol e
6     private static final ThreadLocal<String> TOKEN = new
7         ThreadLocal<>();
8
9     public static void setToken(String token) { TOKEN.set(token); }
10    public static String getToken() { return TOKEN.get(); }
```

```

10 // Imperatif de vider le ThreadLocal a la fin de la requete pour
    // éviter les fuites memoires
11 public static void clearToken() { TOKEN.remove(); }
12 }

```

8.4 L'Implémentation de SessionInterceptor.java

Cet intercepteur s'exécute juste avant qu'une requête HTTP n'atteigne nos contrôleurs Spring. Il extrait le token JWT et alimente le `SessionContext`.

Créez le fichier `src/main/java/com/starterkit/cache/interceptor/SessionInterceptor.java` :

Listing 8.2 – Intercepteur SessionInterceptor.java

```

1 package com.starterkit.cache.interceptor;
2
3 import com.starterkit.cache.context.SessionContext;
4 import jakarta.servlet.http.HttpServletRequest;
5 import jakarta.servlet.http.HttpServletResponse;
6 import org.springframework.stereotype.Component;
7 import org.springframework.web.servlet.HandlerInterceptor;
8
9 @Component
10 public class SessionInterceptor implements HandlerInterceptor {
11
12     @Override
13     public boolean preHandle(HttpServletRequest request,
14                             HttpServletResponse response, Object handler) {
15         SessionContext.clearToken(); // Nettoyage de securite preventif
16         String authHeader = request.getHeader("Authorization");
17         if (authHeader != null && authHeader.startsWith("Bearer ")) {
18             SessionContext.setToken(authHeader.substring(7));
19         }
20         return true;
21     }
22
23     @Override
24     public void afterCompletion(HttpServletRequest request,
25                                HttpServletResponse response,
26                                Object handler, Exception ex) {
27         SessionContext.clearToken(); // Nettoyage propre apres que la
    // requete a ete traitee
28     }
29 }

```

8.5 L'Implémentation de CacheService.java

Créez le fichier `src/main/java/com/starterkit/cache/services/CacheService.java` :

Listing 8.3 – Service Caffeine CacheService.java

```

1 package com.starterkit.cache.services;
2
3 import com.github.benmanes.caffeine.cache.Cache;
4 import com.github.benmanes.caffeine.cache.Caffeine;
5 import com.starterkit.cache.context.SessionContext;
6 import com.starterkit.security.entities.AppSession;

```



```

7 import org.springframework.stereotype.Service;
8 import java.util.concurrent.TimeUnit;
9
10 @Service
11 public class CacheService {
12
13     private final Cache<String, AppSession> sessionCache;
14
15     public CacheService() {
16         // Parametrage du cache Caffeine en memoire
17         sessionCache = Caffeine.newBuilder()
18             .expireAfterWrite(24, TimeUnit.HOURS) // Supprime
19             // l'entree apres 24h d'inactivite
20             .maximumSize(500) // Limite le cache a 500 sessions en
21             // memoire simultanees
22             .build();
23     }
24
25     public void putSession(String token, AppSession session) {
26         sessionCache.put(token, session);
27     }
28
29     public AppSession getSession(String token) {
30         return sessionCache.getIfPresent(token);
31     }
32
33     public AppSession getCurrentSession() {
34         String token = SessionContext.getToken();
35         return (token != null && !token.isBlank()) ? getSession(token)
36             : null;
37     }
38
39     public void evictSession(String token) {
40         sessionCache.invalidate(token); // Invalide (supprime) la
41         // session du cache (ex : deconnexion)
42     }
43 }

```

8.6 Configuration MVC : CacheWebConfig.java

Pour que notre intercepteur fonctionne, il faut l'enregistrer dans la configuration MVC de Spring.

Créez le fichier `src/main/java/com/starterkit/cache/configs/CacheWebConfig.java` :

Listing 8.4 – Configuration MVC CacheWebConfig.java

```

1 package com.starterkit.cache.configs;
2
3 import com.starterkit.cache.interceptor.SessionInterceptor;
4 import lombok.RequiredArgsConstructor;
5 import org.springframework.context.annotation.Configuration;
6 import
7     org.springframework.web.servlet.config.annotation.InterceptorRegistry;
8 import
9     org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
10
11 @Configuration

```

```
10 @RequiredArgsConstructor
11 public class CacheWebConfig implements WebMvcConfigurer {
12
13     private final SessionInterceptor sessionInterceptor;
14
15     @Override
16     public void addInterceptors(InterceptorRegistry registry) {
17         // Enregistre l'intercepteur pour toutes les urls de notre
18         // application
19         registry.addInterceptor(sessionInterceptor).addPathPatterns("/**");
20     }
21 }
```

À retenir en 3 points

1. Le cache Caffeine stocke les données fréquemment lues en RAM pour éviter les allers-retours SQL lents.
2. `ThreadLocal` isole les variables de session d'un thread à l'autre sans risque de mélange d'identité.
3. Vider le `ThreadLocal` dans `afterCompletion` prévient les fuites de mémoire graves.

Chapitre 9

Gestionnaire Global d'Exceptions

9.1 Pourquoi centraliser la gestion des erreurs ?

9.1.1 La Logique Métier (Avant le code)

Imaginez que vous allez dans un restaurant chic. Si la cuisine manque d'ingrédients, le chef ne vient pas crier sur vous en vous jetant une liste de codes techniques incompréhensibles à la figure. Le serveur vient vous voir avec le sourire et vous dit poliment : "Toutes nos excuses, ce plat n'est plus disponible ce soir".

En programmation, quand une erreur survient (un mot de passe incorrect, une donnée obligatoire vide), par défaut, Java lève une exception et affiche une **Stacktrace** (une pile d'erreurs système longue et effrayante). C'est un **danger de sécurité** (elle révèle l'arborescence de vos fichiers, vos tables de base de données) et c'est illisible pour le client mobile ou frontend.

Le gestionnaire global d'exceptions (Global Exception Handler) intercepte toutes les erreurs à la volée pour renvoyer une réponse propre, standardisée et polie au format JSON.

9.2 L'Implémentation de AppException.java

Créez le fichier `src/main/java/com/starterkit/core/exceptions/AppException.java` :

Listing 9.1 – Classe AppException.java

```
1 package com.starterkit.core.exceptions;
2
3 import org.springframework.http.HttpStatus;
4
5 public class AppException extends RuntimeException {
6
7     private final HttpStatus status; // Porte le code statut HTTP (ex:
8         400 BAD REQUEST)
9
10    public AppException(String message, HttpStatus status) {
11        super(message);
12        this.status = status;
13    }
14
15    public HttpStatus getStatus() {
16        return status;
17    }
18 }
```

9.3 L'Implémentation de GlobalExceptionHandler.java

Créez le fichier `src/main/java/com/starterkit/core/exceptions/GlobalExceptionHandler.java` :

Listing 9.2 – Gestionnaire global des exceptions GlobalExceptionHandler.java

```

1 package com.starterkit.core.exceptions;
2
3 import org.slf4j.Logger;
4 import org.slf4j.LoggerFactory;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.MediaType;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.security.access.AccessDeniedException;
9 import
10     org.springframework.security.authentication.AnonymousAuthenticationToken;
11 import
12     org.springframework.security.authorization.AuthorizationDeniedException;
13 import org.springframework.security.core.Authentication;
14 import org.springframework.security.core.AuthenticationException;
15 import org.springframework.security.core.context.SecurityContextHolder;
16 import org.springframework.web.bind.MethodArgumentNotValidException;
17 import org.springframework.web.bind.annotation.ControllerAdvice;
18 import org.springframework.web.bind.annotation.ExceptionHandler;
19 import org.springframework.web.servlet.NoHandlerFoundException;
20 import java.util.HashMap;
21 import java.util.Map;
22 import java.util.stream.Collectors;
23
24 @ControllerAdvice // Annotation qui configure cette classe comme filet
25 // de securite global de l'application
26 public class GlobalExceptionHandler {
27
28     private static final Logger log =
29         LoggerFactory.getLogger(GlobalExceptionHandler.class);
30
31     // 1. Intercepte nos exceptions metier personnalisees
32     @ExceptionHandler(AppException.class)
33     public ResponseEntity<Map<String, Object>>
34         handleAppException(AppException ex) {
35         log.warn("AppException [{}]: {}", ex.getStatus(),
36             ex.getMessage());
37         return buildResponse(ex.getStatus(), ex.getMessage());
38     }
39
40     // 2. Intercepte les erreurs de validation de DTOs (ex : mot de
41     // passe trop court)
42     @ExceptionHandler(MethodArgumentNotValidException.class)
43     public ResponseEntity<Map<String, Object>>
44         handleValidationException(MethodArgumentNotValidException ex) {
45         String errors = ex.getBindingResult().getFieldErrors().stream()
46             .map(fe -> fe.getField() + ": " +
47                 fe.getDefaultMessage())
48             .collect(Collectors.joining(", "));
49         log.warn("Erreur de validation : {}", errors);
50         return buildResponse(HttpStatus.BAD_REQUEST, errors);
51     }
52
53     // 3. Intercepte les erreurs 404 (Route introuvable)

```

```

45 @ExceptionHandler(NoHandlerFoundException.class)
46 public ResponseEntity<Map<String, Object>>
    handleNoHandlerFound(NoHandlerFoundException ex) {
47     log.warn("Route introuvable : {}", ex.getRequestURL());
48     return buildResponse(HttpStatus.NOT_FOUND, "Route introuvable :
        " + ex.getRequestURL());
49 }
50
51 // 4. Intercepte les acces denies Spring Security (401 vs 403)
52 @ExceptionHandler({AuthorizationDeniedException.class,
    AccessDeniedException.class})
53 public ResponseEntity<Map<String, Object>>
    handleAccessDenied(RuntimeException ex) {
54     Authentication auth =
        SecurityContextHolder.getContext().getAuthentication();
55     boolean anonymous = auth == null
56         || auth instanceof AnonymousAuthenticationToken
57         || !auth.isAuthenticated();
58
59     if (anonymous) {
60         log.warn("Acc s refus      non authentifi : {}",
            ex.getMessage());
61         return buildResponse(HttpStatus.UNAUTHORIZED, "Non
            authentifi ou session expir e");
62     } else {
63         log.warn("Acc s refus      droits insuffisants : {}",
            ex.getMessage());
64         return buildResponse(HttpStatus.FORBIDDEN, "Acc s refus
            droits insuffisants");
65     }
66 }
67
68 // 5. Filet de securite ultime pour les erreurs systeme (ex : crash
    de base de donnees)
69 @ExceptionHandler(Exception.class)
70 public ResponseEntity<Map<String, Object>>
    handleGenericException(Exception ex) {
71     log.error("Erreur inattendue : {}", ex.getMessage(), ex); //
        Log la stacktrace complete en interne
72     return buildResponse(HttpStatus.INTERNAL_SERVER_ERROR, "Une
        erreur inattendue est survenue"); // Reponse polie au client
73 }
74
75 private ResponseEntity<Map<String, Object>>
    buildResponse(HttpStatus status, String message) {
76     Map<String, Object> body = new HashMap<>();
77     body.put("status", status.value());
78     body.put("error", message);
79     return ResponseEntity.status(status)
        .contentType(MediaType.APPLICATION_JSON)
        .body(body);
80 }
81
82 }
83 }

```

À retenir en 3 points

1. **@ControllerAdvice** centralise le traitement de toutes les exceptions émises par les contrôleurs.
2. Il ne faut jamais renvoyer de stacktrace brute de Java au client (risque de sécurité).
3. Séparer les logs internes (complets avec stacktrace) des retours clients (JSON formaté et simple).

Chapitre 10

Système de Logging (Logs)

10.1 Pourquoi utiliser des Logs au lieu de `System.out.println` ?

10.1.1 La Logique Métier (Avant le code)

Imaginez la boîte noire d'un avion. Si l'avion subit une avarie, les enquêteurs ont besoin de lire un journal précis décrivant chaque seconde de vol.

En Java, utiliser `System.out.println` est une **très mauvaise pratique** :

- `System.out.println` bloque le thread en cours (c'est une écriture synchrone lente).
- Vous ne pouvez pas désactiver ces messages à moins de supprimer le code source.
- Tous les messages se mélangent sans date, sans provenance, et sans niveau de gravité.

Nous utilisons le framework de journalisation **SLF4J** avec son implémentation **Logback** (inclus d'office dans Spring Boot).

10.1.2 Les Niveaux de Log (Ordre croissant de gravité)

1. **TRACE** : Journalisation ultra-détaillée (étapes internes d'une boucle).
2. **DEBUG** : Informations utiles pour diagnostiquer les bugs (ex : paramètres reçus).
3. **INFO** : Événement normal de la vie de l'application (ex : "Serveur démarré sur le port 8080").
4. **WARN** : Alerte sur une anomalie non bloquante (ex : "Tentative de connexion infructueuse pour l'email X").
5. **ERROR** : Erreur bloquante critique (ex : "Connexion à la base de données perdue").

10.2 Configuration dans `application.properties`

Vous pouvez modifier dynamiquement la verbosité des logs depuis votre `application.properties` :

```
1 # Niveaux de log generaux
2 logging.level.root=INFO
3 # Afficher des requetes SQL detaillées en mode debug
4 logging.level.org.hibernate.SQL=DEBUG
5 # Niveaux de log specifiques a notre Starter-kit
6 logging.level.com.starterkit=DEBUG
```

Dans votre code Java, utilisez Lombok avec l'annotation `@Slf4j` ou déclarez un logger manuellement :

```
1 private static final Logger log =
    LoggerFactory.getLogger(MonService.class);
2 log.info("Traitement en cours pour l'utilisateur {}", email);
```

À retenir en 3 points

1. `System.out.println` dégrade les performances et ne doit pas être utilisé en production.
2. SLF4J / Logback offre des niveaux de journalisation (DEBUG, INFO, WARN, ERROR) configurables.
3. Le formatage dynamique avec accolades `{}` évite les concaténations de chaînes coûteuses en RAM.

Chapitre 11

Test et Validation

11.1 Déroulement pas-à-pas du test de l'API

Pour valider l'implémentation de votre Starter-kit de A à Z, nous vous conseillons d'utiliser des outils de test d'API comme **Postman** ou le site d'auto-documentation **Swagger UI** intégré à l'application et disponible sur <http://localhost:8080/swagger-ui/index.html>.

Voici l'enchaînement des requêtes à tester :

11.1.1 1. Test de l'Inscription (Register)

Envoyez une requête POST sur `/api/auth/register` avec le JSON suivant :

```
1 {  
2   "email": "test@starterkit.com",  
3   "password": "superpassword123",  
4   "fullName": "Jean Starter"  
5 }
```

Résultat attendu : HTTP 200 OK. Vous devez recevoir un objet contenant un `accessToken` de type JWT, un `refreshToken` (UUID), et un temps de validité `expiresIn`.

11.1.2 2. Test de l'Accès Sécurisé

Tentez d'accéder à un endpoint protégé de votre application (ex : `/api/users/me` ou une route factice) sans ajouter de headers. **Résultat attendu** : HTTP 401 Unauthorized, géré proprement par notre `GlobalExceptionHandler`.

11.1.3 3. Ajout de l'Authorization Bearer

Refaites la requête précédente en ajoutant le Header `HTTP : Authorization: Bearer <votre_access_token>`. **Résultat attendu** : HTTP 200 OK avec le contenu demandé.

11.1.4 4. Renouvellement du Token (Refresh)

Une fois les 15 minutes d'Access Token écoulées, ou pour tester de force, envoyez une requête POST sur `/api/auth/refresh` avec le refresh token reçu à l'étape 1 :

```
1 {  
2   "refreshToken": "votre-uuid-de-refresh-token"  
3 }
```

Résultat attendu : HTTP 200 OK. Vous recevez un nouvel `accessToken` tout neuf, ce qui permet à votre utilisateur de rester connecté en toute transparence.

11.1.5 5. Déconnexion (Logout)

Envoyez une requête POST sur `/api/auth/logout` avec le refresh token. **Résultat attendu :** HTTP 204 No Content. La session a été révoquée en base SQL et expulsée du cache Caffeine. Le refresh token est désormais inutilisable.

À retenir en 3 points

1. Swagger UI compile automatiquement nos routes pour nous offrir un bac à sable de test graphique.
2. Le Header **Authorization: Bearer <token>** est nécessaire pour traverser notre filtre de sécurité.
3. Le test d'enchaînement complet (Register → Login → Refresh → Logout) garantit la solidité du Starter-kit.